



ECOLE NORMALE SUPÉRIURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

Département Informatique

Rapport d'examen Architecture Distribuée et Middleware

CONCEPTION ET DÉVELOPPEMENT D'UNE APPLICATION
WEB JEE
POUR LA GESTION DE CRÉDITS BANCAIRES

Réalisé par :
Anass MAKHLOUK

Encadré par :
Prof. YOUSSEF

Masters : SDIA
Module : Architecture Distribuée et Middleware

Année Universitaire : 2025 – 2026

Table des matières

1	Introduction et Objectifs du Projet	3
1.1	Contexte	3
1.2	Objectifs	3
2	Stack Technique	3
3	Architecture Générale	3
3.1	Architecture en couches (Backend)	3
3.2	Architecture Frontend	4
4	Modèle de Données et Héritage JPA	4
4.1	Stratégie d'Héritage : SINGLE_TABLE	4
4.1.1	Entités du domaine	5
4.2	Tables générées par Hibernate	5
5	API REST et Documentation Swagger	7
5.1	Endpoints exposés	7
5.2	Captures Swagger UI	8
6	Sécurité JWT et Gestion des Rôles	11
6.1	Principe de fonctionnement	11
6.2	Comptes de test (hardcodés)	12
6.3	Test d'authentification JWT	13
7	Frontend Angular avec Tailwind CSS	14
7.1	Architecture du projet Angular	14
7.2	Page de Connexion	15
7.3	Dashboard Employé	16
7.4	Espace Client	17
7.5	Espace Admin	18
8	Extraits de Code Majeurs	19
8.1	Entité mère <code>Credit</code> (Héritage SINGLE_TABLE)	19
8.2	Controller REST des Crédits	19
8.3	Service Métier <code>CreditServiceImpl</code>	20
8.4	Configuration de Sécurité Spring Security + JWT	22
8.5	Filtre JWT <code>JwtAuthorizationFilter</code>	23
8.6	Intercepteur HTTP Angular (JWT)	23
8.7	Service d'Authentification Angular	24
9	Organisation en Commits Git	26

10 Tests et Validation	27
10.1 Tests Backend via Swagger	27
10.2 Tests Frontend	27
11 Conclusion	27

1 Introduction et Objectifs du Projet

1.1 Contexte

Ce projet s'inscrit dans le cadre du module *Architecture Distribuée et Middleware* du Master SDIA. Il consiste à concevoir et développer une application Web complète de **gestion de crédits bancaires**, en mettant en œuvre une architecture moderne basée sur une séparation claire entre un backend *RESTful* et un frontend *Single Page Application (SPA)*.

1.2 Objectifs

L'application doit permettre :

- La gestion des **clients** bancaires (consultation, création, suppression).
- La gestion des **crédits** sous trois formes : Personnel, Immobilier et Professionnel.
- Le suivi des **remboursements** associés à chaque crédit.
- Un système d'**authentification stateless** basé sur JSON Web Token (JWT).
- Un contrôle d'accès basé sur les rôles : `ROLE_CLIENT`, `ROLE_EMPLOYE`, `ROLE_ADMIN`.
- Une **documentation API** interactive via Swagger/OpenAPI.
- Un **frontend Angular** avec Tailwind CSS pour une interface moderne et réactive.

2 Stack Technique

Couche	Technologies
Backend	Java 21, Spring Boot 3.3, Spring Web, Spring Data JPA, Spring Security 6
Base de données	H2 (in-memory), Hibernate 6 / JPA
Sécurité	Spring Security, JWT (jjwt 0.12.6), BCryptPasswordEncoder
Documentation	SpringDoc OpenAPI 2.5 / Swagger UI
Frontend	Angular 17, TypeScript, Tailwind CSS 3
Build	Maven (Spring Boot Maven Plugin), npm/Angular CLI 17

TABLE 1 – Stack technique du projet

3 Architecture Générale

3.1 Architecture en couches (Backend)

Le backend suit une architecture en couches strictement séparées :

1. **Couche Entités (JPA)** : Modèle de données avec héritage.
2. **Couche Repository** : Interfaces Spring Data JPA.
3. **Couche Service** : Logique métier (DTOs, Mappers, implémentations).

4. **Couche Web (REST)** : Contrôleurs exposant l'API.
5. **Couche Sécurité** : Filtres JWT, configuration Spring Security.

3.2 Architecture Frontend

Le frontend Angular 17 est structuré en modules fonctionnels :

- **Models** : Interfaces TypeScript miroir du backend.
- **Services** : AuthService, ClientService, CreditService.
- **Interceptors** : JwtInterceptor pour l'injection automatique du token.
- **Guards** : authGuard, employeGuard pour la protection des routes.
- **Pages** : Login, Dashboard Employé (crédits), Dashboard Client (mes crédits), Clients.
- **Shared** : Composant Navbar adaptatif selon le rôle.

4 Modèle de Données et Héritage JPA

4.1 Stratégie d'Héritage : SINGLE_TABLE

Le projet implémente trois types de crédits qui héritent tous d'une classe abstraite **Credit**. La stratégie d'héritage choisie est **SINGLE_TABLE** : une seule table **credits** en base de données contient toutes les colonnes pour les trois sous-types, discriminés par la colonne **TYPE_CREDIT**.

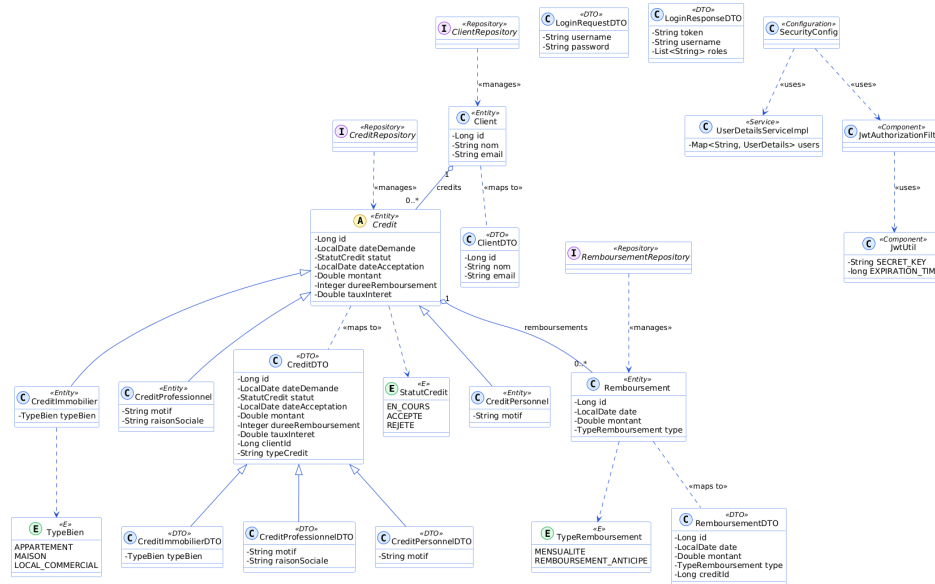


FIGURE 1 – Diagramme de classes UML – Modèle de données

4.1.1 Entités du domaine

Entité	Type	Attributs spécifiques
Credit	Abstraite	id, dateDemande, statut, dateAcceptation, montant, durée, taux, client
CreditPersonnel	Concrète	motif
CreditImmobilier	Concrète	typeBien (APPARTEMENT, MAISON, LOCAL_COMMERCIAL)
CreditProfessionnel	Concrète	motif, raisonSociale
Client	Entité	id, nom, email
Remboursement	Entité	id, date, montant, type (MENSUALITE, REMBOURSEMENT_ANTICIPE)

TABLE 2 – Description des entités JPA

4.2 Tables générées par Hibernate

Les captures suivantes montrent les tables générées automatiquement par Hibernate dans la console H2.

localhost:8080/h2-console/login.do?sessionId=60b286e25b12f030ee690d3f731fc008

Auto commit: ☒ Max rows: 1000 Auto complete: Off Auto select: On

jdbc:h2:mem:creditdb

CLIENTS

CREDITS

REMBOURSEMENTS

INFORMATION_SCHEMA

Users

H2 2.2.224 (2023-09-17)

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM CLIENTS;

ID	EMAIL	NOM
1	ahmed.Ahmed@email.com	Ahmed Ahmed
2	fatima.zahra@email.com	Fatima Zahra
3	youssef.Youssef@email.com	Youssef Youssef
4	sara.Sara@email.com	Sara Sara
5	karim.Karim@email.com	Karim Karim

(5 rows, 3 ms)

Edit

FIGURE 2 – Table `clients` dans la console H2

Auto commit: ☒ Max rows: 1000 Auto complete: Off Auto select: On

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM CREDITS;

TYPE_CREDIT	ID	DATE_ACCEPTION	DATE_DEMANDE	DUREE_REMBOURSEMENT	MONTANT	STATUT	TAUX_INTERET	TYPE_BIEN	MOTIF	RAISON_SOCIALE	CLIENT_ID
PERSONNEL	1	2024-01-25	2024-01-15	24	50000.0	ACCEPTÉ	4.5	null	Achat voiture	null	1
PERSONNEL	2	null	2024-06-10	12	20000.0	EN_COURS	5.0	null	Voyage et formation	null	2
IMMOBILIER	3	2023-09-20	2023-09-01	240	800000.0	ACCEPTÉ	3.2	APPARTEMENT	null	null	3
IMMOBILIER	4	null	2024-05-20	300	1500000.0	REJETÉ	3.8	MAISON	null	null	4
PROFESSIONNEL	5	2024-03-18	2024-03-05	60	300000.0	ACCEPTÉ	4.0	null	Extension activité	Karim Import Export	5
PROFESSIONNEL	6	null	2024-07-12	36	150000.0	EN_COURS	4.8	null	Achat équipement	Ahmed Tech	1

(6 rows, 0 ms)

Edit

FIGURE 3 – Table `credits` avec la colonne discriminante `TYPE_CREDIT` (stratégie `SINGLE_TABLE`)

Auto commit: ☒ Max rows: 1000 Auto complete: Off Auto select: On

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM REMBOURSEMENTS

ID	DATE	MONTANT	TYPE	CREDIT_ID
1	2024-02-25	2200.0	MENSUALITE	1
2	2024-03-25	2200.0	MENSUALITE	1
3	2024-04-25	10000.0	REMBOURSEMENT_ANTICIPE	1
4	2023-10-20	3500.0	MENSUALITE	3
5	2023-11-20	3500.0	MENSUALITE	3
6	2024-04-18	5200.0	MENSUALITE	5
7	2024-05-18	5200.0	MENSUALITE	5

(7 rows, 1 ms)

Edit

FIGURE 4 – Table `remboursements` avec clé étrangère vers `credits`

5 API REST et Documentation Swagger

5.1 Endpoints exposés

L'API REST expose 11 endpoints répartis sur 3 controllers :

Méthode	URL	Rôle requis	Description
POST	/api/auth/login	Public	Connexion et génération du token JWT
GET	/api/clients	EMPLOYE, ADMIN	Lister tous les clients
GET	/api/clients/{id}	EMPLOYE, ADMIN	Obtenir un client par ID
POST	/api/clients	ADMIN	Créer un client
DELETE	/api/clients/{id}	ADMIN	Supprimer un client
GET	/api/credits	EMPLOYE, ADMIN	Lister tous les crédits
GET	/api/credits/{id}	Tous	Obtenir un crédit par ID
GET	/api/clients/{id}/credits	Tous	Crédits d'un client
GET	/api/credits/statut/{statut}	EMPLOYE, ADMIN	Filtrer par statut
POST	/api/credits	Tous	Créer une demande de crédit
PUT	/api/credits/{id}/statut	EMPLOYE, ADMIN	Mettre à jour le statut

TABLE 3 – Liste des endpoints REST

5.2 Captures Swagger UI

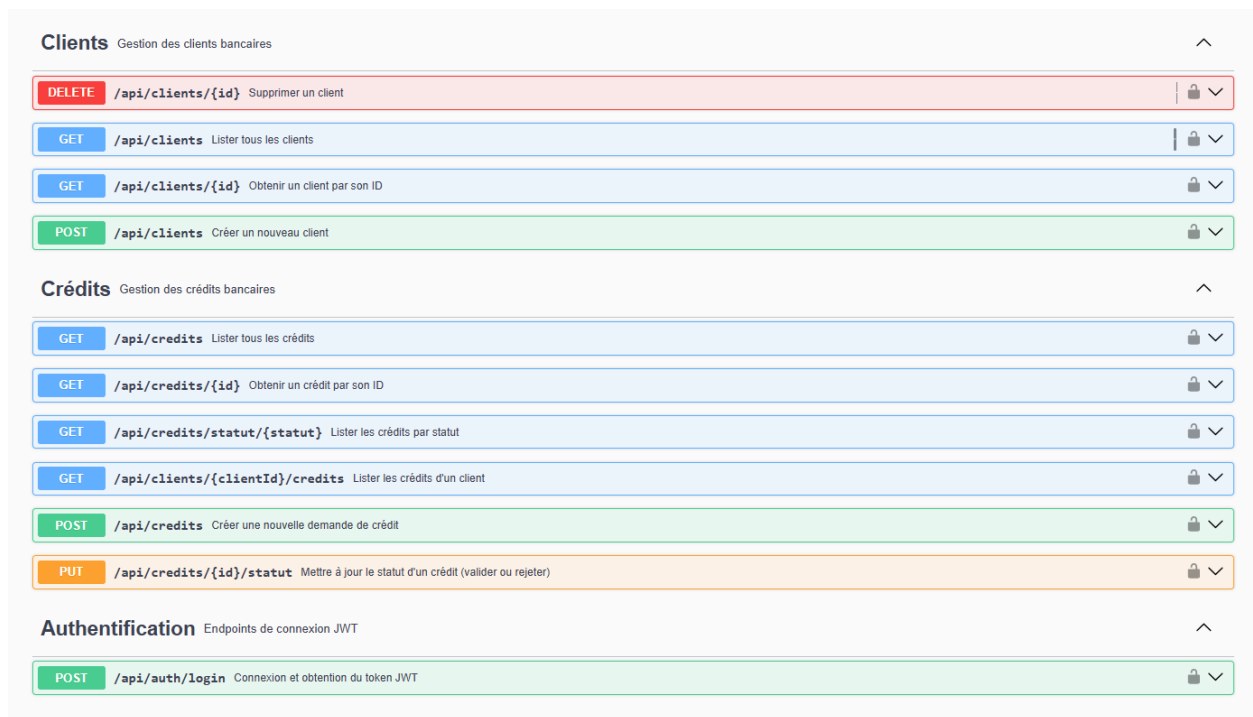


FIGURE 5 – Vue globale de l’API dans Swagger UI – tous les endpoints groupés par controller

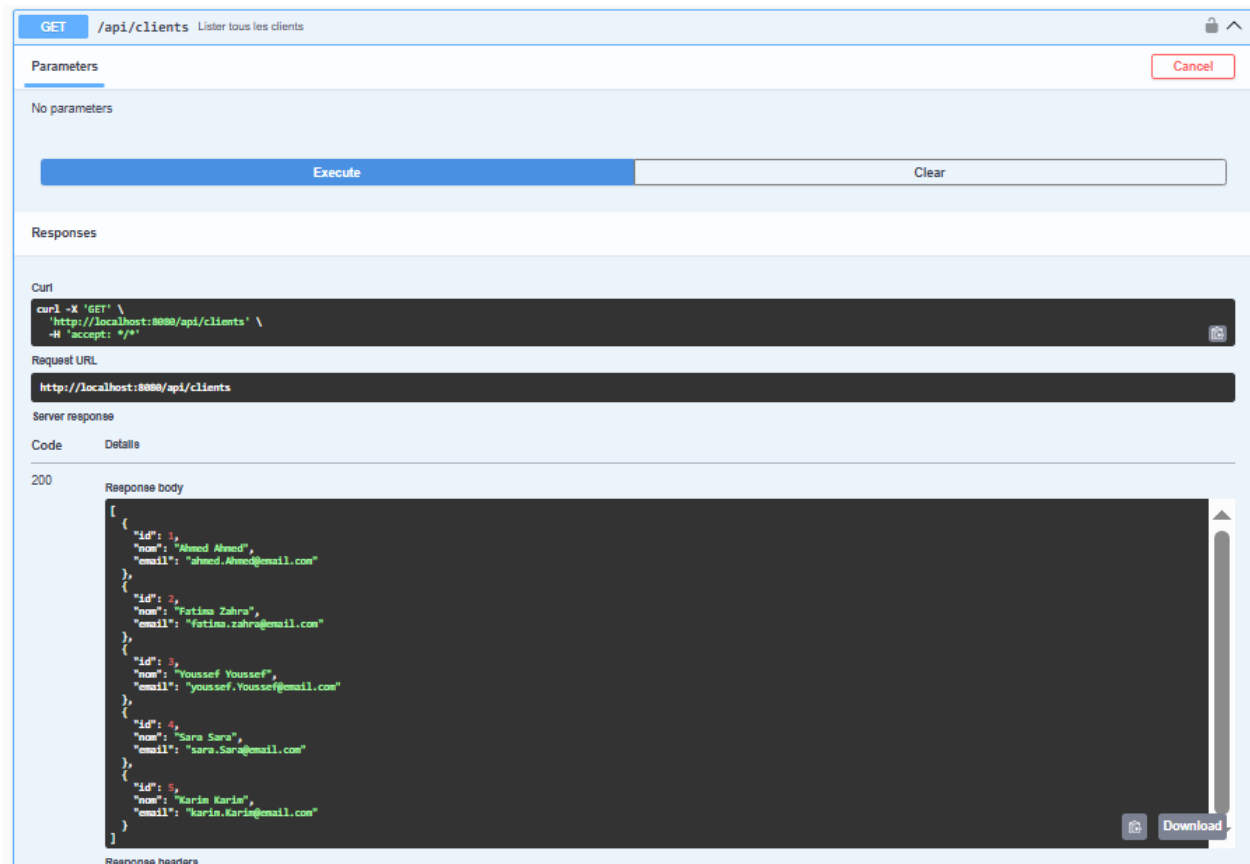


FIGURE 6 – Test GET /api/clients – Récupération de la liste des clients

The screenshot shows a REST client interface with the following sections:

- GET /api/clients/{clientId}/credits** Lister les crédits d'un client
- Parameters**
 - Name**: **clientId** (required, Integer(\$int64), path)
 - Description**: 1
- Execute** button
- Responses**
 - Code**: 200
 - Details**: Response body

The response body contains the following JSON data:

```
[
  {
    "id": 1,
    "dateDemande": "2024-01-15",
    "statut": "ACCEPTÉ",
    "dateAcceptation": "2024-01-25",
    "montant": 50000,
    "dureeRemboursement": 24,
    "tauxInteret": 4.5,
    "clientId": 1,
    "typeCredit": "PERSONNEL",
    "motif": "Achat voiture"
  },
  {
    "id": 6,
    "dateDemande": "2024-07-12",
    "statut": "EN COURS",
    "dateAcceptation": null,
    "montant": 150000,
    "dureeRemboursement": 36,
    "tauxInteret": 4.8,
    "clientId": 1,
    "typeCredit": "PROFESSIONNEL",
    "motif": "Achat équipement",
    "raisonSociale": "Ahmed Tech"
  }
]
```

FIGURE 7 – Test GET /api/clients/{id}/credits – Crédits d'un client spécifique

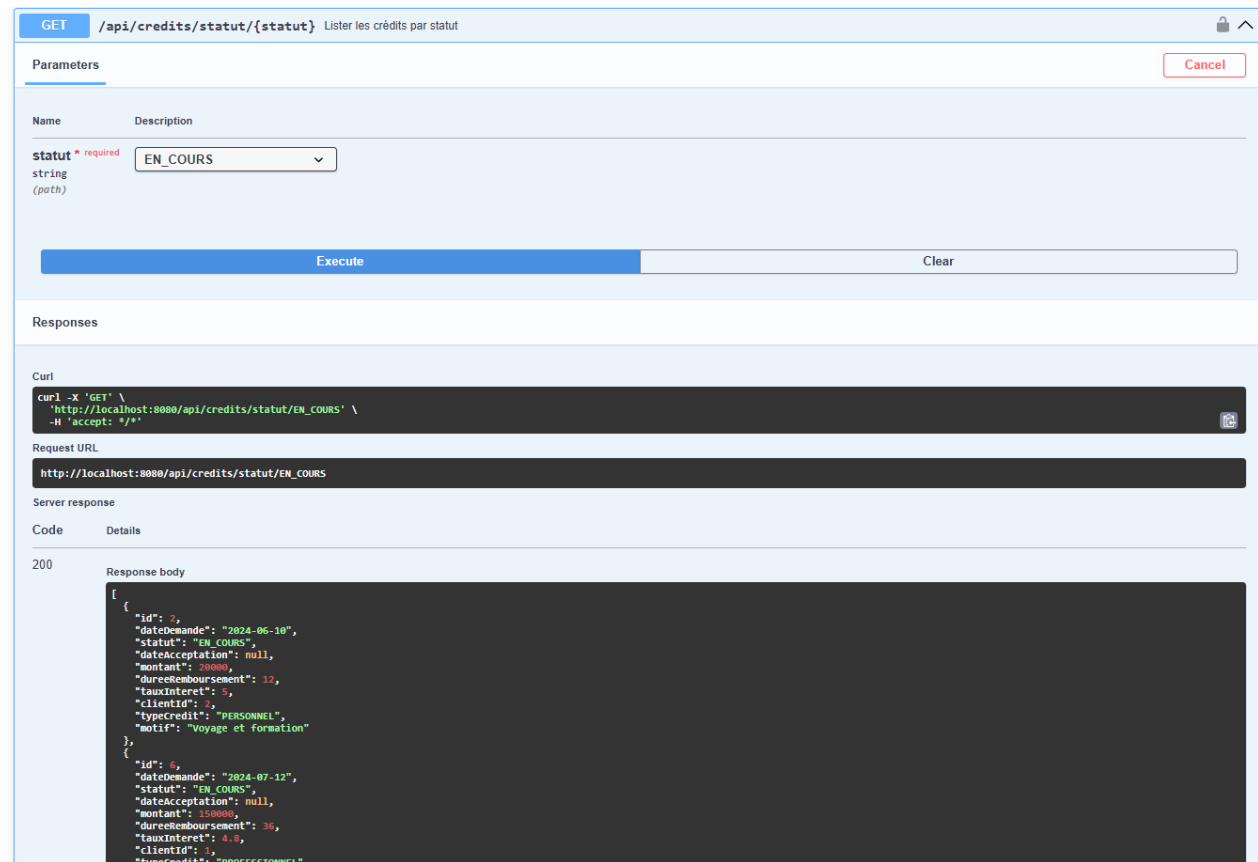


FIGURE 8 – Test GET /api/credits/statut/EN_COURS – Filtrage par statut

6 Sécurité JWT et Gestion des Rôles

6.1 Principe de fonctionnement

L'authentification est **stateless** : aucune session n'est maintenue côté serveur. Le flux est le suivant :

1. Le client envoie ses identifiants à POST /api/auth/login.
2. Spring Security authentifie via DaoAuthenticationProvider + BCrypt.
3. En cas de succès, JwtUtil génère un token signé (algorithme HMAC-SHA).
4. Le token est retourné au client avec la liste des rôles.
5. Chaque requête suivante doit inclure le header Authorization: Bearer <token>.
6. JwtAuthorizationFilter valide le token et charge le SecurityContext.

6.2 Comptes de test (hardcodés)

Utilisateur	Mot de passe	Rôle
client1	secret1234	ROLE_CLIENT
employe1	secret1234	ROLE_EMPLOYE
admin	secret1234	ROLE_ADMIN

TABLE 4 – Comptes de test hardcodés

6.3 Test d'authentification JWT

Authentification Endpoints de connexion JWT

POST /api/auth/login Connexion et obtention du token JWT

Parameters

No parameters

Request body *required* application/json

```
{
  "username": "employe1",
  "password": "secret1234"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "employe1",
    "password": "secret1234"
  }'
```

Request URL

```
http://localhost:8080/api/auth/login
```

Server response

Code	Details
200	Response body <pre>{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTQ5LmVudC01IiwiaWF0IjE1OTUzMTUzInQ9ZS3dCjpyYXQ1OjE3ODAsImV4cCI6MTY0MTAwMzEwMH0.Sn9KMP_FFWs-YvGPr-b-ajBIwHKRUz5vxb3U1FprKvU1skUN9NFTP-V-y9EPH", "username": "employe1", "roles": ["ROLE_EMPLOYE"] }</pre> Response headers <pre>cache-control: no-cache, no-store, max-age=0, must-revalidate connection: keep-alive content-type: application/json date: Mon, 08 Jun 2026 11:05:00 GMT expires: 0 keep-alive: timeout=60 pragma: no-cache transfer-encoding: chunked vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers x-content-type-options: nosniff x-xss-protection: 0</pre>

FIGURE 9 – Génération du token JWT via POST /api/auth/login

PUT /api/credits/{id}/statut Mettre à jour le statut d'un crédit (valider ou rejeter)

Parameters

Cancel

Name	Description
id * required integer(\$int64) (path)	2
statut * required string (query)	ACCEPTÉ

Execute Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:8080/api/credits/2/statut?statut=ACCEPTÉ' \
  -H 'accept: */*'
```

Request URL

http://localhost:8080/api/credits/2/statut?statut=ACCEPTÉ

Server response

Code	Details
200	Response body <pre>{ "id": 2, "dateDemande": "2024-06-18", "statut": "ACCEPTÉ", "dateAcceptation": "2026-06-08", "montant": 20000, "dureeRemboursement": 12, "tauxInteret": 5, "clientId": 1, "typeCredit": "PERSONNEL", "motif": "Voyage et formation" }</pre> Response headers <pre>cache-control: no-cache,no-store,max-age=0,must-revalidate connection: keep-alive content-type: application/json date: Mon, 08 Jun 2026 10:48:27 GMT expires: 0</pre>

FIGURE 10 – Mise à jour du statut d'un crédit par un employé (token JWT en header)

7 Frontend Angular avec Tailwind CSS

7.1 Architecture du projet Angular

Le frontend est un projet Angular 17 standalone (sans NgModules), utilisant :

- **Lazy Loading** pour chaque page (amélioration des performances).
- **Functional Guards** (authGuard, employeGuard) pour la protection des routes.
- **Functional Interceptors** (jwtInterceptor) pour l'injection automatique du Bearer token.
- **Tailwind CSS** pour le design avec classes utilitaires.

7.2 Page de Connexion

COMPTES DE TEST	
client1	CLIENT
employe1	EMPLOYÉ
admin	ADMIN

Mot de passe : secret1234

FIGURE 11 – Mire de connexion Angular – Gradient bleu, formulaire réactif, affichage des comptes de test

La page de connexion utilise un **formulaire réactif** Angular avec validation et affiche une table des comptes disponibles pour faciliter les tests. En cas de succès, la redirection est automatique selon le rôle :

- `ROLE_CLIENT` → `/mes-credits`
- `ROLE_EMPLOYE` ou `ROLE_ADMIN` → `/credits`

7.3 Dashboard Employé

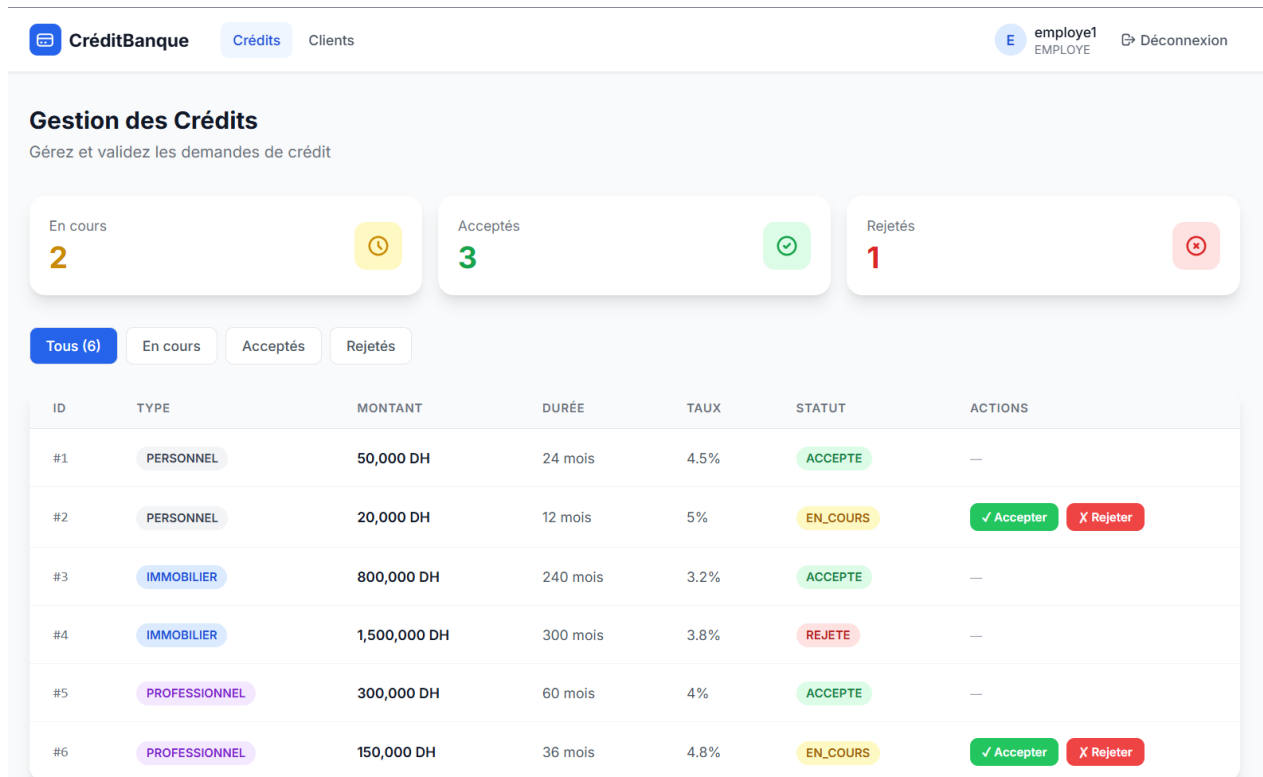
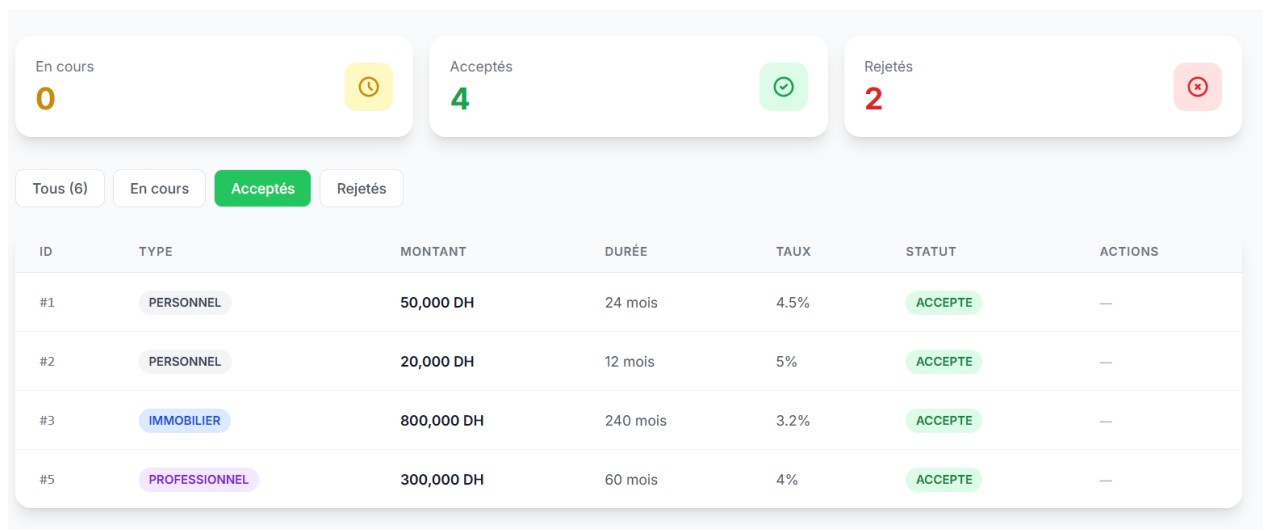


FIGURE 12 – Dashboard Employé – KPIs de statuts, filtres, tableau avec boutons Accepter/Rejeter



Le dashboard employé affiche :

- 3 cartes KPI : nombre de crédits EN_COURS, ACCEPTÉS, REJETÉS.
- Un système de **filtres** par statut.

- Un tableau complet avec les boutons **Accepter** et **Rejeter** qui appellent PUT /api/credits/{id}/st

7.4 Espace Client



FIGURE 13 – Espace Client – Visualisation des crédits en cartes avec statut coloré

7.5 Espace Admin

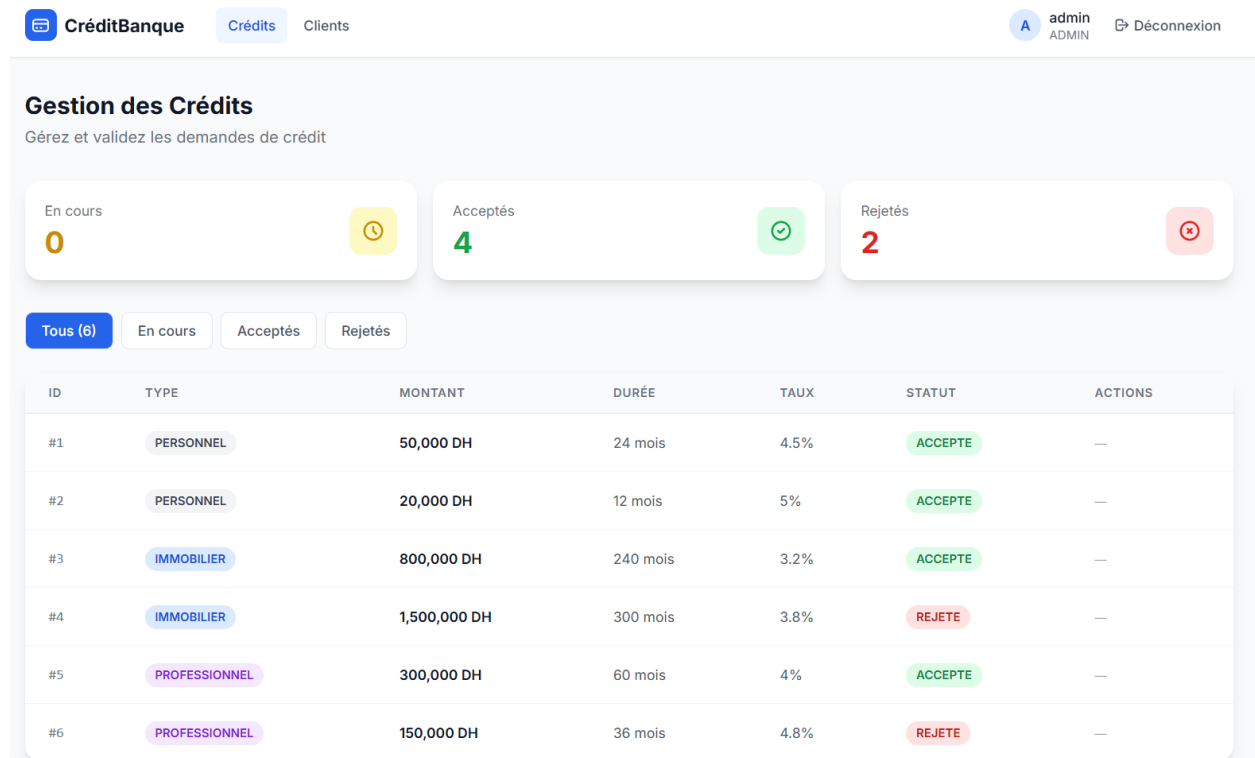


FIGURE 14 – Espace Admin

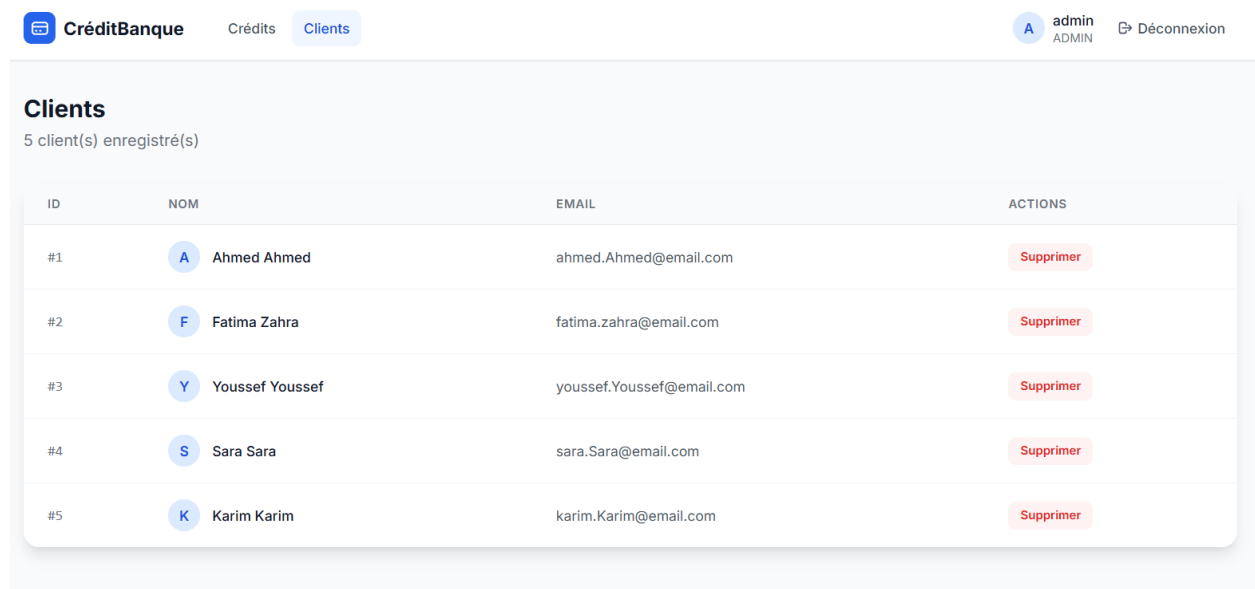


FIGURE 15 – Espace Admin - gestion clients

8 Extraits de Code Majeurs

8.1 Entité mère Credit (Héritage SINGLE_TABLE)

Listing 1 – Credit.java – Entité abstraite avec stratégie SINGLE_TABLE

```
1 @Entity
2 @Table(name = "credits")
3 @Inheritance(strategy = InheritanceType.SINGLE_TABLE)
4 @DiscriminatorColumn(name = "TYPE_CREDIT", discriminatorType =
5     DiscriminatorType.STRING)
6 @Data
7 @NoArgsConstructor
8 @AllArgsConstructor
9 @SuperBuilder
10 public abstract class Credit {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     private LocalDate dateDemande;
16
17     @Enumerated(EnumType.STRING)
18     private StatutCredit statut;
19
20     private LocalDate dateAcceptation;
21     private Double montant;
22     private Integer dureeRemboursement;
23     private Double tauxInteret;
24
25     @ManyToOne
26     @JoinColumn(name = "client_id")
27     private Client client;
28
29     @OneToMany(mappedBy = "credit", cascade = CascadeType.ALL)
30     private List<Remboursement> remboursements;
31 }
```

8.2 Controller REST des Crédits

Listing 2 – CreditController.java – Endpoints REST

```
1 @RestController
2 @RequiredArgsConstructor
3 @Tag(name = "Credits", description = "Gestion des credits bancaires")
4 public class CreditController {
```

```
5
6     private final CreditService creditService;
7
8     @GetMapping("/api/credits")
9     @Operation(summary = "Lister tous les credits")
10    public ResponseEntity<List<CreditDTO>> getAllCredits() {
11        return ResponseEntity.ok(creditService.getAllCredits());
12    }
13
14    @GetMapping("/api/clients/{clientId}/credits")
15    @Operation(summary = "Lister les credits d un client")
16    public ResponseEntity<List<CreditDTO>> getCreditsByClient(
17        @PathVariable Long clientId) {
18        return ResponseEntity.ok(
19            creditService.getCreditsByClientId(clientId));
20    }
21
22    @GetMapping("/api/credits/statut/{statut}")
23    @Operation(summary = "Filtrer les credits par statut")
24    public ResponseEntity<List<CreditDTO>> getCreditsByStatut(
25        @PathVariable StatutCredit statut) {
26        return ResponseEntity.ok(
27            creditService.getCreditsByStatut(statut));
28    }
29
30    @PostMapping("/api/credits")
31    @Operation(summary = "Creer une demande de credit")
32    public ResponseEntity<CreditDTO> createCredit(
33        @RequestBody CreditDTO creditDTO) {
34        return ResponseEntity.status(HttpStatus.CREATED)
35            .body(creditService.saveCredit(creditDTO));
36    }
37
38    @PutMapping("/api/credits/{id}/statut")
39    @Operation(summary = "Mettre a jour le statut d un credit")
40    public ResponseEntity<CreditDTO> updateStatut(
41        @PathVariable Long id,
42        @RequestParam StatutCredit statut) {
43        return ResponseEntity.ok(
44            creditService.updateStatut(id, statut));
45    }
46 }
```

8.3 Service Métier CreditServiceImpl

Listing 3 – CreditServiceImpl.java – Logique metier avec switch Java 21

```

1 @Service
2 @Transactional
3 @RequiredArgsConstructor
4 public class CreditServiceImpl implements CreditService {
5
6     private final CreditRepository creditRepository;
7     private final ClientRepository clientRepository;
8     private final CreditMapper creditMapper;
9
10    @Override
11    public CreditDTO updateStatut(Long id, StatutCredit statut) {
12        Credit credit = creditRepository.findById(id)
13            .orElseThrow(() ->
14                new EntityNotFoundException("Credit non trouve : " +
15                    id));
16
17        credit.setStatut(statut);
18        if (statut == StatutCredit.ACCEPTE) {
19            credit.setDateAcceptation(LocalDate.now());
20        }
21        return creditMapper.toDTO(creditRepository.save(credit));
22    }
23
24    @Override
25    public CreditDTO saveCredit(CreditDTO dto) {
26        Client client = clientRepository.findById(dto.getClientId())
27            .orElseThrow(() ->
28                new EntityNotFoundException("Client non trouve"));
29
30        Credit credit = switch (dto.getTypeCredit().toUpperCase()) {
31            case "PERSONNEL" -> {
32                CreditPersonnelDTO cp = (CreditPersonnelDTO) dto;
33                yield CreditPersonnel.builder()
34                    .dateDemande(LocalDate.now())
35                    .statut(StatutCredit.EN_COURS)
36                    .montant(dto.getMontant())
37                    .dureeRemboursement(dto.getDureeRemboursement())
38                    .tauxInteret(dto.getTauxInteret())
39                    .client(client).motif(cp.getMotif()).build();
40            }
41            case "IMMOBILIER" -> {
42                CreditImmobilierDTO ci = (CreditImmobilierDTO) dto;
43                yield CreditImmobilier.builder()
44                    .dateDemande(LocalDate.now())
45                    .statut(StatutCredit.EN_COURS)
46                    .montant(dto.getMontant())
47                    .dureeRemboursement(dto.getDureeRemboursement())
48                    .tauxInteret(dto.getTauxInteret())

```

```

48         .client(client).typeBien(ci.getTypeBien()).build
49         ();
50     }
51     default -> throw new IllegalArgumentException(
52         "Type inconnu : " + dto.getTypeCredit());
53 };
54 return creditMapper.toDTO(creditRepository.save(credit));
55 }

```

8.4 Configuration de Sécurité Spring Security + JWT

Listing 4 – SecurityConfig.java – Configuration Spring Security stateless

```

1 @Configuration
2 @EnableWebSecurity
3 @RequiredArgsConstructor
4 public class SecurityConfig {
5
6     private final JwtAuthorizationFilter jwtAuthorizationFilter;
7     private final UserDetailsServiceImpl userDetailsService;
8
9     @Bean
10    public SecurityFilterChain filterChain(HttpSecurity http)
11        throws Exception {
12        http
13            .csrf(AbstractHttpConfigurer::disable)
14            .cors(cors -> cors.configurationSource(
15                corsConfigurationSource()))
16            .sessionManagement(session -> session
17                .sessionCreationPolicy(
18                    SessionCreationPolicy.STATELESS))
19            .authorizeHttpRequests(auth -> auth
20                .requestMatchers("/api/auth/**").permitAll()
21                .requestMatchers("/swagger-ui/**",
22                    "/v3/api-docs/**").permitAll()
23                .requestMatchers(HttpMethod.GET, "/api/clients")
24                    .hasAnyRole("EMPLOYE", "ADMIN")
25                .requestMatchers(HttpMethod.DELETE,
26                    "/api/clients/**").hasRole("ADMIN")
27                .requestMatchers(HttpMethod.PUT,
28                    "/api/credits/*/statut")
29                    .hasAnyRole("EMPLOYE", "ADMIN")
30                .anyRequest().authenticated())
31            .addFilterBefore(jwtAuthorizationFilter,
32                UsernamePasswordAuthenticationFilter.class);
33        return http.build();

```

```

34     }
35 }

```

8.5 Filtre JWT JwtAuthorizationFilter

Listing 5 – JwtAuthorizationFilter.java – Filtre d'extraction et validation du token

```

1  @Component
2  @RequiredArgsConstructor
3  public class JwtAuthorizationFilter extends OncePerRequestFilter {
4
5      private final JwtUtil jwtUtil;
6
7      @Override
8      protected void doFilterInternal(HttpServletRequest request,
9                                     HttpServletResponse response, FilterChain filterChain)
10         throws ServletException, IOException {
11
12         String authHeader = request.getHeader("Authorization");
13
14         if (authHeader != null && authHeader.startsWith("Bearer ")) {
15             String token = authHeader.substring(7);
16
17             if (jwtUtil.validateToken(token)) {
18                 String username = jwtUtil.extractUsername(token);
19                 List<String> roles = jwtUtil.extractRoles(token);
20
21                 List<SimpleGrantedAuthority> authorities =
22                     roles.stream()
23                         .map(SimpleGrantedAuthority::new)
24                         .collect(Collectors.toList());
25
26                 UsernamePasswordAuthenticationToken auth =
27                     new UsernamePasswordAuthenticationToken(
28                         username, null, authorities);
29
30                 SecurityContextHolder.getContext()
31                     .setAuthentication(auth);
32             }
33         }
34         filterChain.doFilter(request, response);
35     }
36 }

```

8.6 Intercepteur HTTP Angular (JWT)

Listing 6 – jwt.interceptor.ts – Injection automatique du Bearer token

```

1 import { HttpInterceptorFn } from '@angular/common/http';
2 import { inject } from '@angular/core';
3 import { AuthService } from '../services/auth.service';
4
5 export const jwtInterceptor: HttpInterceptorFn = (req, next) => {
6   const authService = inject(AuthService);
7   const token = authService.getToken();
8
9   if (token) {
10    const cloned = req.clone({
11      setHeaders: { Authorization: `Bearer ${token}` }
12    });
13    return next(cloned);
14  }
15
16  return next(req);
17 };

```

8.7 Service d'Authentification Angular

Listing 7 – auth.service.ts – Gestion JWT cote Angular

```

1 @Injectable({ providedIn: 'root' })
2 export class AuthService {
3   private readonly API = 'http://localhost:8080/api/auth';
4   private readonly TOKEN_KEY = 'jwt_token';
5   private readonly USER_KEY = 'current_user';
6
7   constructor(private http: HttpClient, private router: Router) {}
8
9   login(credentials: LoginRequest): Observable<LoginResponse> {
10    return this.http.post<LoginResponse>(
11      `${this.API}/login`, credentials).pipe(
12      tap((response: LoginResponse) => {
13        localStorage.setItem(this.TOKEN_KEY, response.token);
14        localStorage.setItem(this.USER_KEY,
15          JSON.stringify(response));
16      })
17    );
18  }
19
20  logout(): void {
21    localStorage.removeItem(this.TOKEN_KEY);
22    localStorage.removeItem(this.USER_KEY);
23    this.router.navigate(['/login']);
24  }

```

```
25
26  isEmploye(): boolean {
27      return this.hasRole('ROLE_EMPLOYE') || this.isAdmin();
28  }
29
30  isAdmin(): boolean {
31      return this.hasRole('ROLE_ADMIN');
32  }
33  }
```

9 Organisation en Commits Git

Commits on Jun 8, 2026

add client and employee dashboards with layout navbar

 vnvss-0x committed 1 hour ago

init angular, tailwind, http services, jwt security and login

 vnvss-0x committed 1 hour ago

implement JWT authentication and role-based authorization

 vnvss-0x committed 1 hour ago

configure Swagger et OpenAPI

 vnvss-0x committed 2 hours ago

add REST controllers and expose API endpoints

 vnvss-0x committed 2 hours ago

add DTOs, mappers, services

 vnvss-0x committed 2 hours ago

add database seeder with test data

 vnvss-0x committed 3 hours ago

add JPA entities with SINGLE_TABLE inheritance

 vnvss-0x committed 3 hours ago

add JPA entities with SINGLE_TABLE inheritance

 vnvss-0x committed 3 hours ago

first commit

 vnvss-0x committed 3 hours ago

Le projet a été développé de manière incrémentale en suivant une approche orientée commits :

Commit Description
Initialisation Spring Boot, pom.xml, application.properties 5 entités JPA + enums + héritage SINGLE_TABLE 3 interfaces Repository Spring Data JPA DataInitializer – 5 clients, 6 crédits, 7 remboursements DTOs, Mappers manuels, CreditService/ClientService 3 REST Controllers – 11 endpoints Configuration Swagger/OpenAPI publique JWT, UserDetailsService, JwtFilter, SecurityConfig, CORS Angular 17 + Tailwind, Services HTTP, Intercepteur JWT, Guard, Login Navbar, Dashboard Employé (Accepter/Rejeter), Dashboard Client

TABLE 5 – Organisation des commits du projet

10 Tests et Validation

10.1 Tests Backend via Swagger

Tous les endpoints ont été testés via l’interface Swagger UI accessible à <http://localhost:8080/swagger>

- **Authentification** : Génération du token JWT avec chaque compte de test.
- **Contrôle d’accès** : Vérification que les routes protégées retournent 403 Forbidden sans token.
- **CRUD Clients** : Création, consultation, suppression.
- **Crédits** : Création des trois types, filtrage par statut, mise à jour du statut.
- **Héritage** : Vérification de la colonne TYPE_CREDIT dans la table `credits`.

10.2 Tests Frontend

- **Login** : Redirection automatique selon le rôle après connexion réussie.
- **Interception JWT** : Présence du header `Authorization` dans toutes les requêtes HTTP.
- **Guard** : Redirection vers `/login` si accès non autorisé.
- **Dashboard Employé** : Affichage des crédits, filtrage par statut, boutons Accepter/-Rejeter fonctionnels.
- **Dashboard Client** : Affichage en cartes avec code couleur par statut.

11 Conclusion

Ce projet a permis de mettre en pratique l’ensemble des concepts du module *Architecture Distribuée et Middleware* :

- La conception d'une **API REST** robuste avec Spring Boot 3.
- L'implémentation de **l'héritage JPA** avec la stratégie **SINGLE_TABLE**.
- La sécurisation stateless via **JSON Web Token** et Spring Security 6.
- Le développement d'un **frontend moderne** avec Angular 17 et Tailwind CSS.
- La **séparation des préoccupations** à travers une architecture en couches stricte.

L'application finale est pleinement fonctionnelle : le backend expose une API documentée et sécurisée, le frontend offre une expérience utilisateur adaptative selon les rôles, et les deux communiquent de manière sécurisée via JWT.